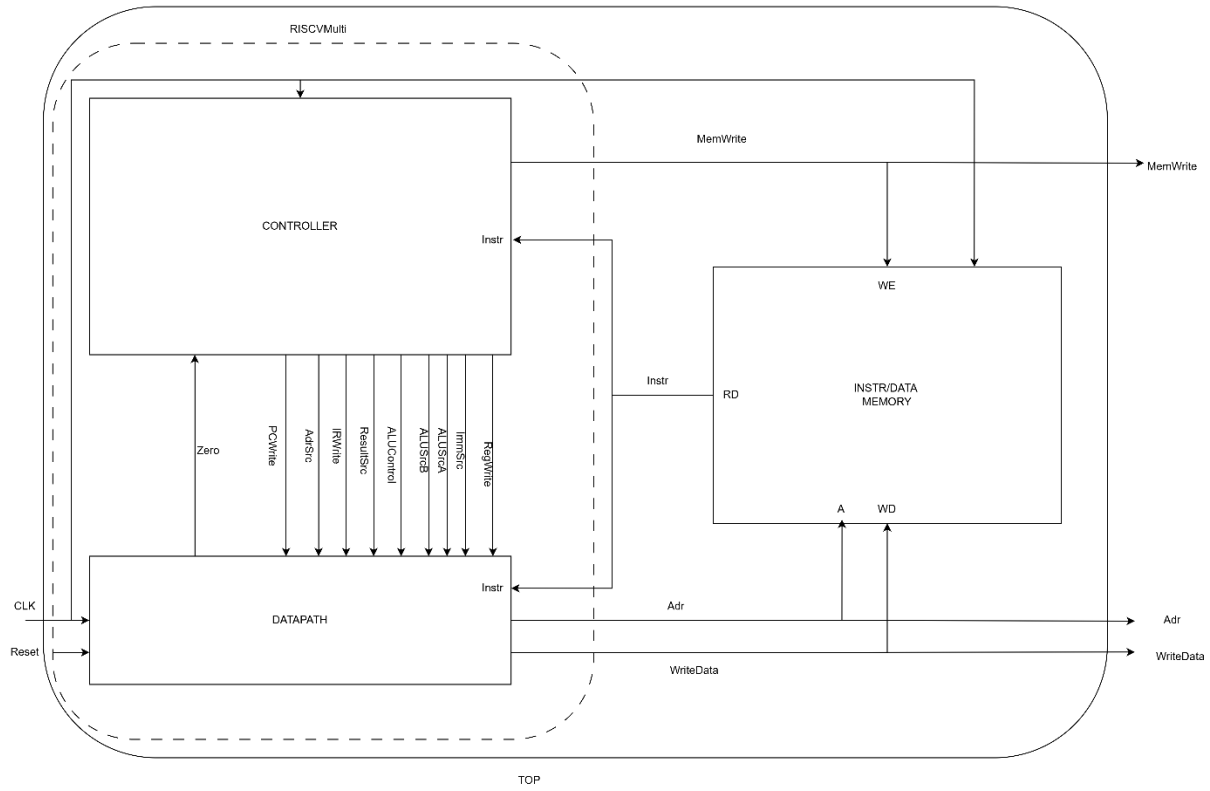
ELE432
Advanced Digital Design
Preliminary Work 3
Multicycle Processor RISC-V

Name: Emre Kerim Kaymak Number: 2220357045

Q1) HOURS SPENT

Approximately 5 hours has been spent on this homework, 2 hours of writing the code, 1 hour of debugging, I was quite lucky that the code worked in the first iteration this time. The remaining 2 hours were used for drawing schematics as well and also to prepare this document.

Q2) HIERARCHICAL DIAGRAM



```
module top(
    input  logic clk, reset,
    output logic [31:0] WriteData, DataAdr,
    output logic MemWrite
);

    // ================= INTERNAL SIGNALS =================

    // PC path
    logic [31:0] PC, PCNext, OldPC;

    // Memory
    logic [31:0] Adr, ReadData, Data;

    // Instruction
    logic [31:0] Instr;

    // Register file
    logic [31:0] RD1, RD2;
    logic [31:0] RD1_reg, RD2_reg;

    // ALU
    logic [31:0] SrcA, SrcB, ALUResult, ALUOut;
    logic Zero;

    // Immediate
    logic [31:0] ImmExt;

    // Result
    logic [31:0] Result;

    // Control signals
    logic [1:0] ImmSrc, ALUSrcA, ALUSrcB, ResultSrc;
    logic [2:0] ALUControl;
    logic AdrSrc;
    logic IRWrite, PCWrite, RegWrite;

    // ================= CONTROLLER =================
    controller ctrl (
        .clk(clk),
        .reset(reset),
        .op(Instr[6:0]),
        .funct3(Instr[14:12]),
        .funct7b5(Instr[30]),
        .zero(Zero),

        .immsrc(ImmSrc),
        .alusrca(ALUSrcA),
        .alusrcb(ALUSrcB),
        .resultsrc(ResultSrc),
        .adrsrc(AdrSrc),
        .alucontrol(ALUControl),
        .irwrite(IRWrite),
        .pcwrite(PCWrite),
        .regwrite(RegWrite),
        .memwrite(MemWrite)
    );
```

```
// ================= PC REGISTER =================
always_ff @(posedge clk or posedge reset) begin
    if (reset)
        PC <= 32'b0;
    else if (PCWrite)
        PC <= PCNext;
end

// ================= OLD PC REGISTER =================
always_ff @(posedge clk) begin
    if (IRWrite)
        OldPC <= PC;
end

// ================= ADDRESS MUX =================
assign Adr = (AdrSrc) ? Result : PC;

// ================= MEMORY =================
mem memory (
    .clk(clk),
    .WE(MemWrite),
    .A(Adr),
    .WD(RD2_reg),
    .RD(ReadData)
);

// ================= INSTRUCTION REGISTER ================
always_ff @(posedge clk) begin
    if (IRWrite)
        Instr <= ReadData;
end

// ================= DATA REGISTER =================
always_ff @(posedge clk) begin
    Data <= ReadData;
end

// ================= REGISTER FILE =================
regfile rf (
    .clk(clk),
    .reset(reset),
    .WE3(RegWrite),
    .A1(Instr[19:15]),
    .A2(Instr[24:20]),
    .A3(Instr[11:7]),
    .WD3(Result),
    .RD1(RD1),
    .RD2(RD2)
);

// ================= RD1 / RD2 PIPE REGISTERS =================
always_ff @(posedge clk) begin
    RD1_reg <= RD1;
    RD2_reg <= RD2;
end

// ================= IMMEDIATE =================
sign_extend imm (
    .instr(Instr),
    .Immsrc(ImmSrc),
    .dataout(ImmExt)
);
```

```
  // ================= SRC A MUX =================
    always_comb begin
        case (ALUSrcA)
            2'b00: SrcA = PC;
            2'b01: SrcA = OldPC;
            2'b10: SrcA = RD1_reg;
            default: SrcA = 32'b0;
        endcase
    end

    // ================= SRC B MUX =================
    always_comb begin
        case (ALUSrcB)
            2'b00: SrcB = RD2_reg;
            2'b01: SrcB = ImmExt;   // ← your correction
            2'b10: SrcB = 32'd4;    // ← your correction
            default: SrcB = 32'b0;
        endcase
    end

    // ================= ALU =================
    myALU alu (
        .SrcA(SrcA),
        .SrcB(SrcB),
        .ALUControl(ALUControl),
        .zerof(Zero),
        .ALUResult(ALUResult)
    );

    // ================= ALU OUTPUT REGISTER =================
    always_ff @(posedge clk) begin
        ALUOut <= ALUResult;
    end

    // ================= RESULT MUX =================
    always_comb begin
        case (ResultSrc)
            2'b00: Result = ALUOut;
            2'b01: Result = Data;
            2'b10: Result = ALUResult;
            default: Result = 32'b0;
        endcase
    end

    // ================= PC NEXT =================
    assign PCNext = Result;

    // ================= OUTPUTS =================
    assign WriteData = RD2_reg;
    assign DataAdr   = Adr;

endmodule
```

```
module regfile (
    input  logic        clk,
    input  logic        reset,
    input  logic        WE3,
    input  logic [4:0]  A1,
    input  logic [4:0]  A2,
    input  logic [4:0]  A3,
    input  logic [31:0] WD3,
    output logic [31:0] RD1,
    output logic [31:0] RD2
);

    logic [31:0] rf [31:0];


    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            for (int i = 0; i < 32; i++)
                rf[i] <= 32'b0;
        end
        else if (WE3 && (A3 != 5'b0)) begin
            rf[A3] <= WD3;
        end
    end


    always_comb begin
        RD1 = (A1 == 5'b0) ? 32'b0 : rf[A1];
        RD2 = (A2 == 5'b0) ? 32'b0 : rf[A2];
    end

endmodule



module sign_extend (
    input  logic [31:0] instr,
    input  logic [1:0]  Immsrc,
    output logic [31:0] dataout
);
    always_comb begin
        case (Immsrc)
            2'b00:  // I-type
                dataout = {{20{instr[31]}}, instr[31:20]};
            2'b01:  // S-type
                dataout = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            2'b10:  // B-type
                dataout = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};
            2'b11:  // J-type
                dataout = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};
            default:
                dataout = 32'b0;
        endcase
    end
endmodule
```

```
module controller(
    input  logic clk,
    input  logic reset,
    input  logic [6:0] op,
    input  logic [2:0] funct3,
    input  logic funct7b5,
    input  logic zero,

    output logic [1:0] immsrc,
    output logic [1:0] alusrca, alusrcb,
    output logic [1:0] resultsrc,
    output logic adrsrc,
    output logic [2:0] alucontrol,
    output logic irwrite, pcwrite,
    output logic regwrite, memwrite
);

    // ========= OPCODE CONSTANTS =========
    localparam [6:0]
        LW  = 7'b0000011,
        SW  = 7'b0100011,
        RT  = 7'b0110011,
        IT  = 7'b0010011,
        JAL = 7'b1101111,
        BEQ = 7'b1100011;





    // ========= STATE TYPE =========
    typedef enum logic [3:0] {
        FETCH,
        DECODE,
        MEMADR,
        MEMREAD,
        MEMWB,
        MEMWRITE,
        EXECUTER,
        ALUWB,
        EXECUTEL,
        JAL_S,
        BEQ_S
    } state_t;

    state_t state_cur, state_next;

    logic [1:0] ALUOp;

    // ALU DECODER
    aludec alu_decoder (
        .opb5(op[5]),
        .funct3(funct3),
        .funct7b5(funct7b5),
        .ALUOp(ALUOp),
        .ALUControl(alucontrol)
    );

   //INSTR DECODER
       instrdec instr_decoder (
        .op(op),
        .ImmSrc(immsrc)
    );
```

```
// ================= STATE REGISTER =================
  always_ff @(posedge clk or posedge reset) begin
      if (reset)
          state_cur <= FETCH;
      else
          state_cur <= state_next;
  end

  // ================= NEXT STATE LOGIC =================
  always_comb begin
      state_next = state_cur;

      case (state_cur)

          FETCH:    state_next = DECODE;

          DECODE: begin
              case (op)
                  LW, SW: state_next = MEMADR;
                  RT:     state_next = EXECUTER;
                  IT:     state_next = EXECUTEL;
                  JAL:    state_next = JAL_S;
                  BEQ:    state_next = BEQ_S;
                  default:state_next = FETCH;
              endcase
          end

          MEMADR: begin
              if (op == LW) state_next = MEMREAD;
              else          state_next = MEMWRITE;
          end

          MEMREAD:   state_next = MEMWB;
          MEMWB:     state_next = FETCH;
          MEMWRITE:  state_next = FETCH;

          EXECUTER:  state_next = ALUWB;
          EXECUTEL:  state_next = ALUWB;
          BEQ_S:     state_next = FETCH;
          ALUWB:     state_next = FETCH;
          JAL_S:     state_next = ALUWB;

      endcase
  end
```

```
// ================= OUTPUT LOGIC =================
  always_comb begin
      // ===== DEFAULTS =====
      //immsrc    = 2'b00;
      alusrca   = 2'b00;
      alusrcb   = 2'b00;
      resultsrc = 2'b00;
      adrsrc    = 1'b0;


      irwrite   = 1'b0;
      pcwrite   = 1'b0;
      regwrite  = 1'b0;
      memwrite  = 1'b0;

      ALUOp     = 2'b00;
     /*
      // ===== IMM SRC based on opcode =====
      case (op)
          LW, IT: immsrc = 2'b00;
          SW:     immsrc = 2'b01;
          BEQ:    immsrc = 2'b10;
          JAL:    immsrc = 2'b11;
          default: immsrc = 2'b00;
      endcase
     */




      // ===== STATE-BASED OUTPUTS =====
      case (state_cur)

          FETCH: begin
              alusrca = 2'b00;
              alusrcb = 2'b10;
              irwrite = 1'b1;
              pcwrite = 1'b1;
              resultsrc = 2'b10;
              adrsrc = 1'b0;
              ALUOp = 2'b00;
          end

          DECODE: begin
              alusrca = 2'b01;
              alusrcb = 2'b01;
              ALUOp = 2'b00;
          end

          MEMADR: begin
              alusrca = 2'b10;
              alusrcb = 2'b01;
              ALUOp = 2'b00;
          end

          MEMREAD: begin
              adrsrc = 1'b1;
              resultsrc = 2'b00;
          end
```

```
            MEMWB: begin
                resultsrc = 2'b01;
                regwrite = 1'b1;
            end

            MEMWRITE: begin
                resultsrc = 2'b00;
                adrsrc = 1'b1;
                memwrite = 1'b1;
            end

            EXECUTER: begin
                alusrca = 2'b10;
                alusrcb = 2'b00;
                ALUOp = 2'b10;
            end

            EXECUTEL: begin
                alusrca = 2'b10;
                alusrcb = 2'b01;
                ALUOp = 2'b10;
            end

            ALUWB: begin
                resultsrc = 2'b00;
                regwrite = 1'b1;
            end

            JAL_S: begin
                alusrca = 2'b01;
                alusrcb = 2'b10;
                ALUOp = 2'b00;
                resultsrc = 2'b00;
                pcwrite = 1'b1;
            end

            BEQ_S: begin
                alusrca = 2'b10;
                alusrcb = 2'b00;
                ALUOp = 2'b01;
                resultsrc = 2'b00;
                if (zero)
                    pcwrite = 1'b1;
            end

        endcase
    end

endmodule
```

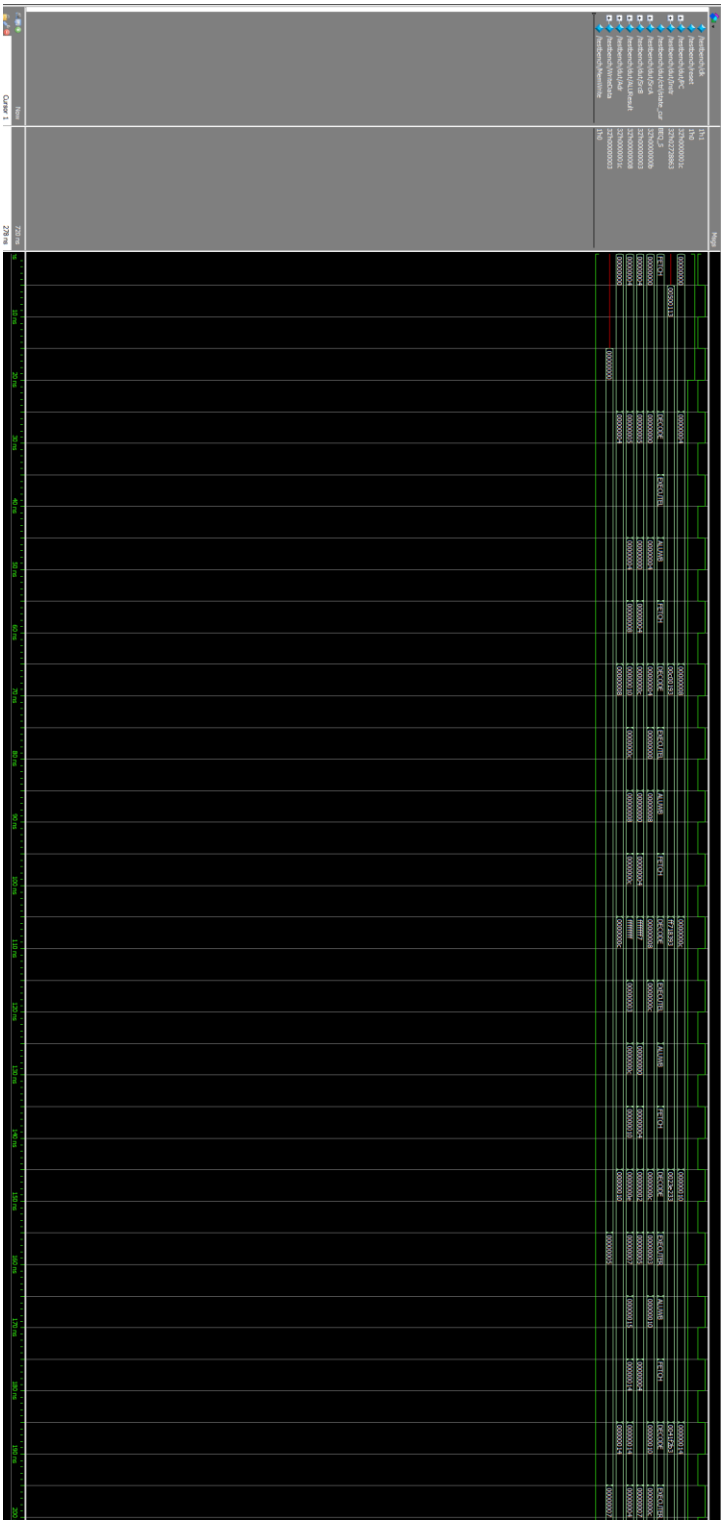
```
module aludec(input logic opb5,
input logic [2:0] funct3,
input logic funct7b5,
input logic [1:0] ALUOp,
output logic [2:0] ALUControl);
logic RtypeSub;
assign RtypeSub = funct7b5 & opb5; // TRUE for R-type subtract instruction
always_comb
case(ALUOp)
2'b00: ALUControl = 3'b000; // addition
2'b01: ALUControl = 3'b001; // subtraction
default: case(funct3) // R-type or I-type ALU
3'b000: if (RtypeSub)
ALUControl = 3'b001; // sub
else
ALUControl = 3'b000; // add, addi
3'b010: ALUControl = 3'b101; // slt, slti
3'b110: ALUControl = 3'b011; // or, ori
3'b111: ALUControl = 3'b010; // and, andi
default: ALUControl = 3'bxxx; // ???
endcase
endcase
endmodule

module instrdec(
    input logic [6:0] op,
    output logic [1:0] ImmSrc
);

    always_comb
    case(op)
        7'b0110011: ImmSrc = 2'b00; // R-type
        7'b0010011: ImmSrc = 2'b00; // I-type ALU
        7'b0000011: ImmSrc = 2'b00; // lw
        7'b0100011: ImmSrc = 2'b01; // sw
        7'b1100011: ImmSrc = 2'b10; // beq
        7'b1101111: ImmSrc = 2'b11; // jal
        default: ImmSrc = 2'b00;
    endcase

endmodule

module mem (
    input  logic        clk,
    input  logic        WE,       // write enable
    input  logic [31:0] A,        // address
    input  logic [31:0] WD,       // write data
    output logic [31:0] RD        // read data
);

    logic [31:0] RAM [0:63];

    // Initialize memory from file
    initial begin
        $readmemh("memfile.txt", RAM);
    end

    // Write (synchronous)
    always_ff @(posedge clk) begin
        if (WE) begin
            RAM[A[31:2]] <= WD;
        end
    end

    // Read (asynchronous)
    assign RD = RAM[A[31:2]];

endmodule
```

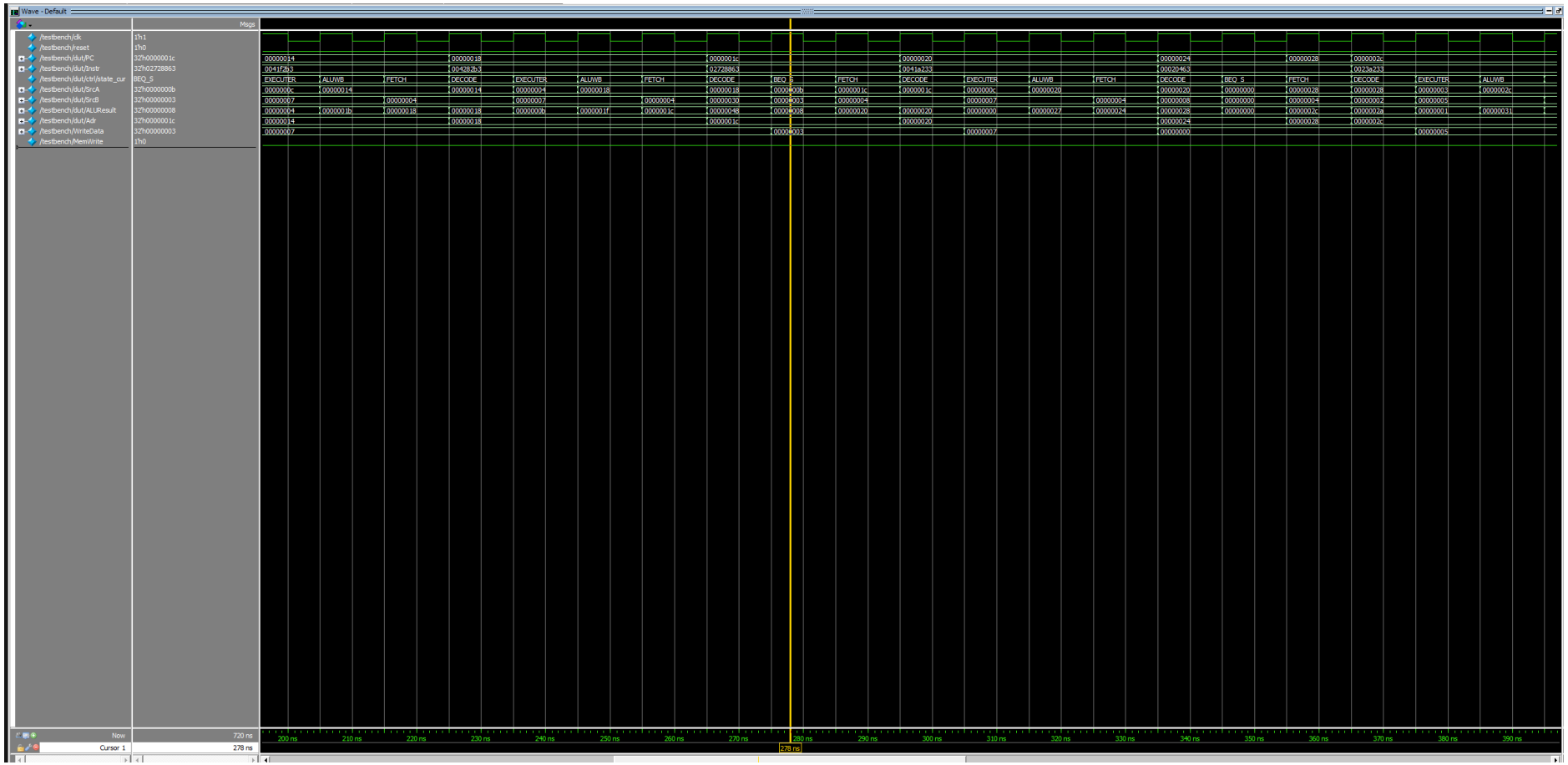
```
module adder_n #(
    parameter N = 32
) (
    input  logic [N-1:0] a,
    input  logic [N-1:0] b,
    input  logic         cin,
    output logic [N-1:0] sum,
    output logic         cout
);
    assign {cout, sum} = a + b + cin;
endmodule


module myALU #(
    parameter N = 32
)(
    input  logic [31:0] SrcA,
    input  logic [31:0] SrcB,
    input  logic [2:0]  ALUControl,
    output logic        zerof,
    output logic [31:0] ALUResult
);

    logic [31:0] Bmux;
    logic [31:0] sum;
    logic        cout;
    logic        overflow;
    logic        cin;

    assign Bmux = SrcB ^ {32{ALUControl[0]}}; //basically take invert if 1 o.w. 0
    assign cin  = ALUControl[0];

    adder_n #(.N(32)) adder_inst (
        .a   (SrcA),
        .b   (Bmux),
        .cin (cin),
        .sum (sum),
        .cout(cout)
    );
    assign overflow =
        (ALUControl[1] == 1'b0) &&
        (SrcA[31] == Bmux[31]) &&
        (sum[31]  != SrcA[31]);

    // ALU Operations

    always_comb begin
        case (ALUControl)

            3'b000: ALUResult = sum;                      // ADD
            3'b001: ALUResult = sum;                      // SUB
            3'b010: ALUResult = SrcA & SrcB;              // AND
            3'b011: ALUResult = SrcA | SrcB;              // OR
            3'b100: ALUResult = SrcA ^ SrcB;              // XOR

            3'b101: // SLT (signed)
                ALUResult = {31'b0, (overflow ^ sum[31])};

            3'b110: ALUResult = SrcA >> SrcB[4:0];        // SRL
            3'b111: ALUResult = SrcA << SrcB[4:0];        // SLL

            default: ALUResult = 32'b0;
        endcase

        zerof = (ALUResult == 32'b0);
    end

endmodule
```

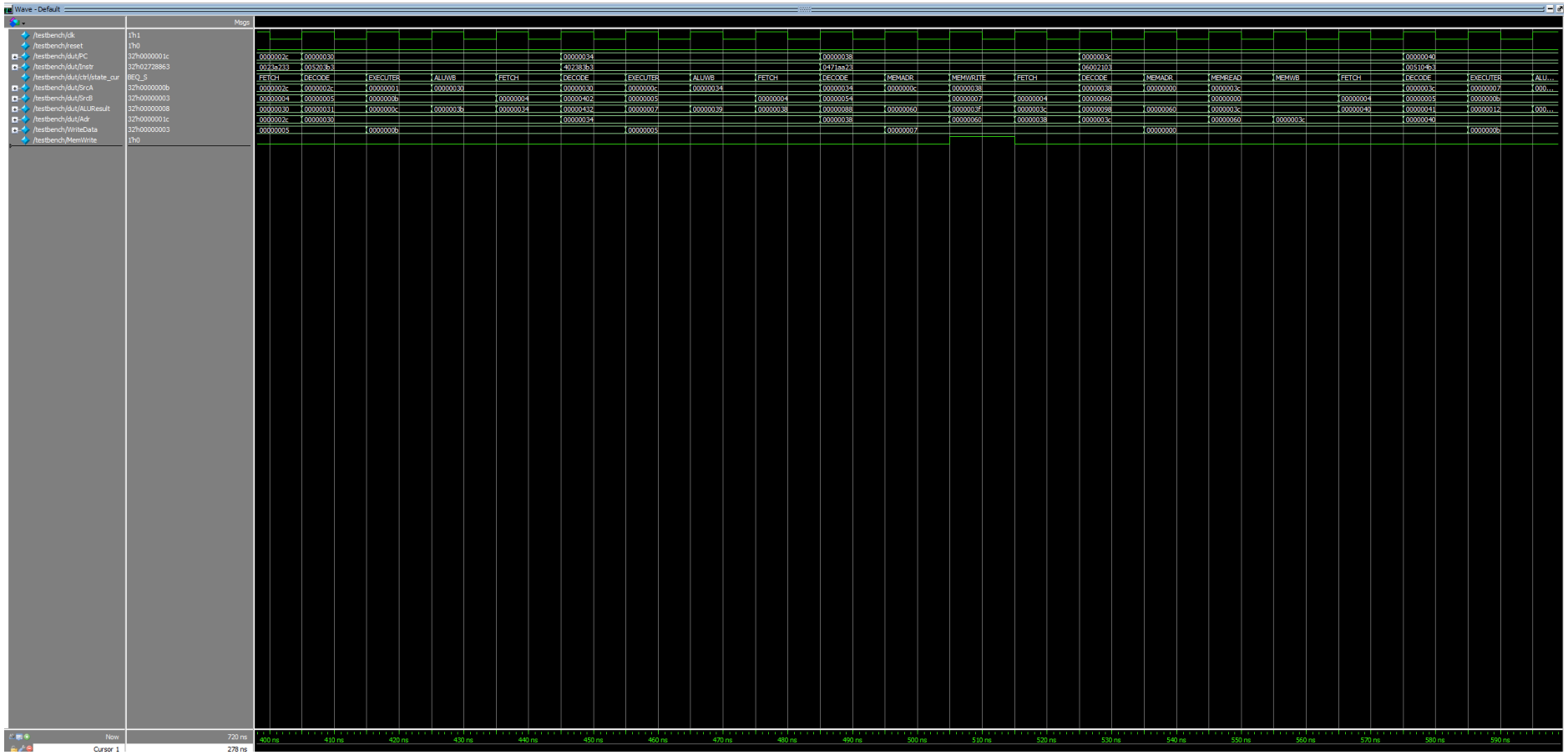
Q4) KEY SIGNALS FOR FIRST 3 INSTRUCTIONS

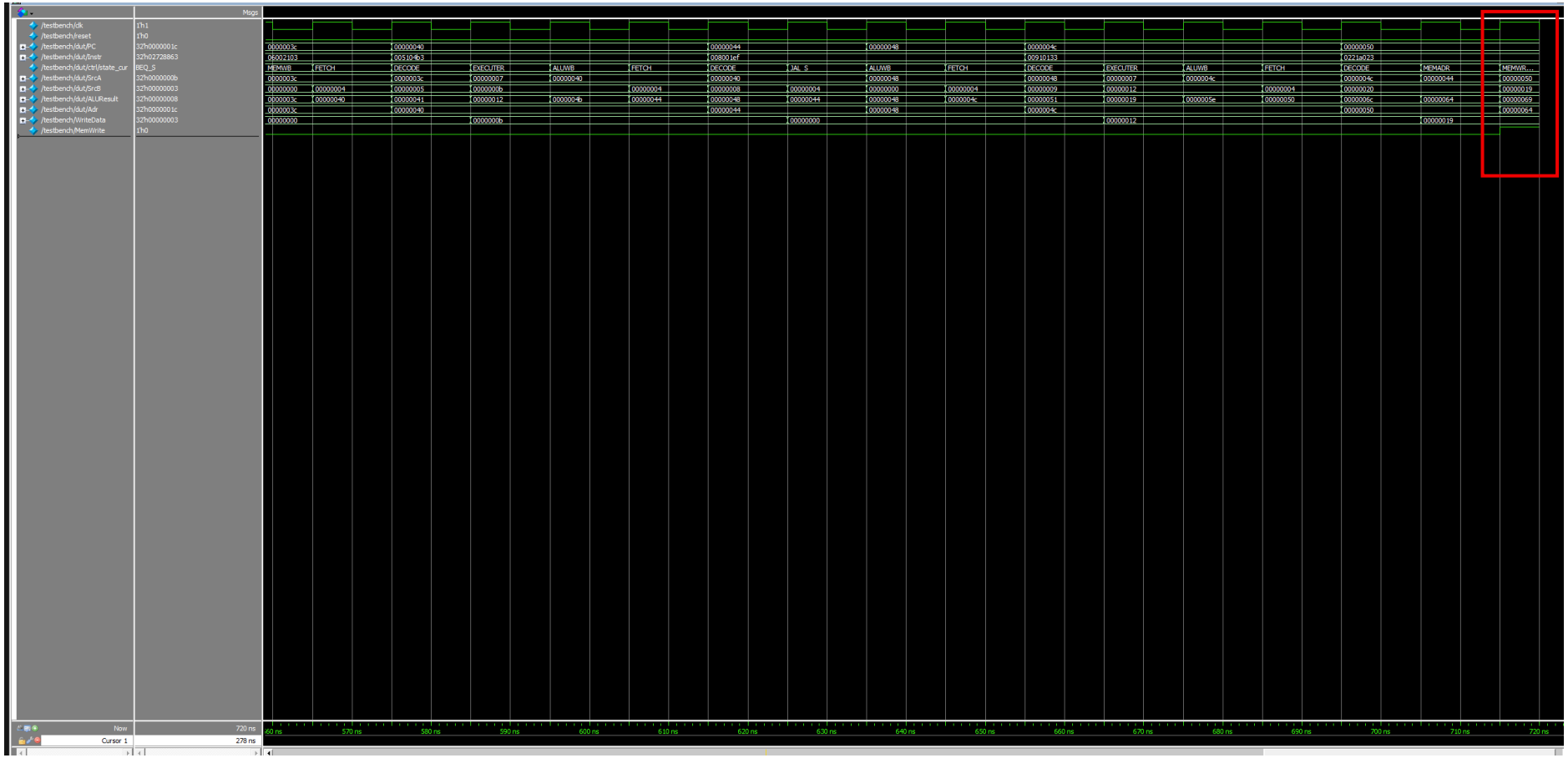
STEP	PC	INSTR	STATE	RESULT (in hex)
3	00	00500113	FETCH	4
4	04	00500113	DECODE	X
5	04	00500113	EXECUTEL	X
6	04	00500113	ALUWB	5
7	04	00500113	FETCH	8
8	08	00C00193	DECODE	X
9	08	00C00193	EXECUTEL	X
10	08	00C00193	ALUWB	C
11	08	00C00193	FETCH	C
12	0C	FF718393	DECODE	X
13	0C	FF718393	EXECUTEL	X
14	0C	FF718393	ALUWB	3
15	10	FF718393	FETCH	10

Q5) SIMULATION WAVEFORMS




Wave - Default
Msgs
/testbench/clk 1'h1
/testbench/reset 1'h0
/testbench/dut/PC 32'h0000001c
/testbench/dut/Instr 32'h02728863
/testbench/dut/ctrl/state_cur BEQ_S
/testbench/dut/SrcA 32'h0000000b
/testbench/dut/SrcB 32'h00000003
/testbench/dut/ALUResult 32'h00000008
/testbench/dut/Adr 32'h0000001c
/testbench/WriteData 32'h00000003
/testbench/MemWrite 1'h0
Now 720 ns
Cursor 1 278 ns
200 ns 210 ns 220 ns 230 ns 240 ns 250 ns 260 ns 270 ns 280 ns 290 ns 300 ns 310 ns 320 ns 330 ns 340 ns 350 ns 360 ns 370 ns 380 ns 390 ns

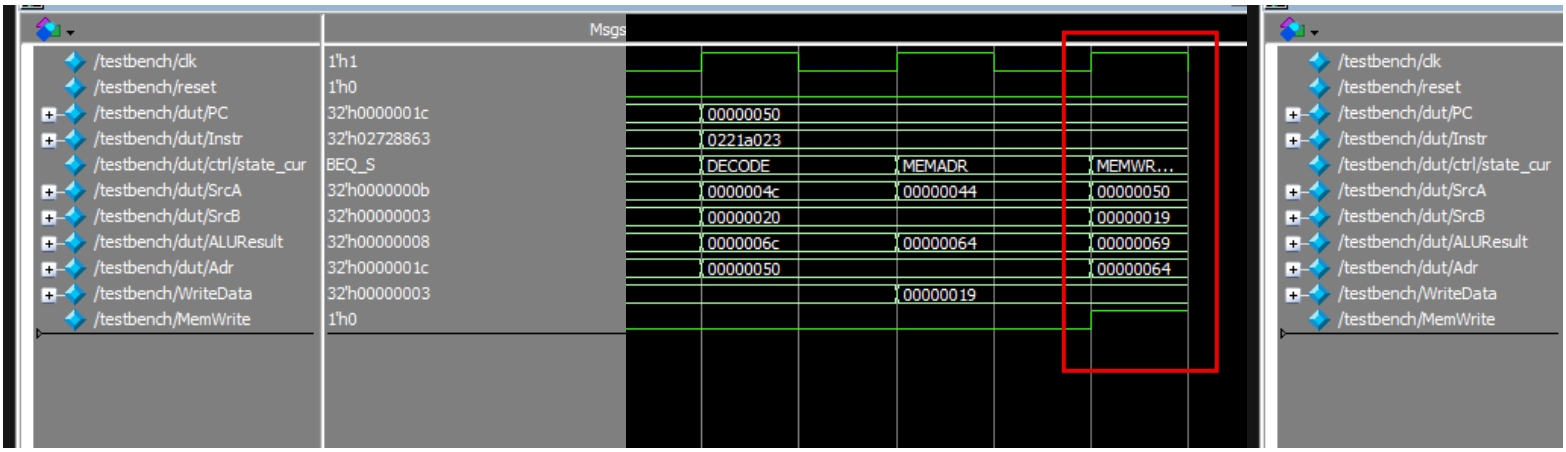

Wave - Default
Msgs
/testbench/clk 1'h1
/testbench/reset 1'h0
/testbench/dut/PC 32'h0000001c
/testbench/dut/Instr 32'h02728863
/testbench/dut/ctrl/state_cur BEQ_S
/testbench/dut/SrcA 32'h0000000b
/testbench/dut/SrcB 32'h00000003
/testbench/dut/ALUResult 32'h00000008
/testbench/dut/Adr 32'h0000001c
/testbench/WriteData 32'h00000003
/testbench/MemWrite 1'h0
Now 720 ns
Cursor 1 278 ns


Msgs
/testbench/clk 1'h1
/testbench/reset 1'h0
/testbench/dut/PC 32'h0000001c
/testbench/dut/Instr 32'h02728863
/testbench/dut/ctrl/state_cur BEQ_S
/testbench/dut/SrcA 32'h0000000b
/testbench/dut/SrcB 32'h00000003
/testbench/dut/ALUResult 32'h00000008
/testbench/dut/Adr 32'h0000001c
/testbench/WriteData 32'h00000003
/testbench/MemWrite 1'h0
Now 720 ns
Cursor 1 278 ns

At the end of the last page, the confirmation that the test vector is correct can be checked, as given is the riscvtest.s file "# If successful, it should write the value 25 to address 100", here it can clearly be seen that this is correct.

A zoomed in part of the page is given:

A slight problem is that every instruction seem to happen a PC in delay meaning at a PC + 4 of what is the required by the file. Otherwise, no other serious problem can be observed.



Q6) GITHUB LINK

https://github.com/TM-LEONARDO/RISC_V-MULTICYCLE-PROCESSOR_-PRELIMINARY/tree/main